

Lab - Encrypting and Decrypting Data Using OpenSSL (THY DAYUTH)

Objectives

Part 1: Encrypting Messages with OpenSSL

Part 2: Decrypting Messages with OpenSSL

Background / Scenario

OpenSSL is an open-source project that provides a robust, commercial-grade, and full-featured toolkit for the Transport Layer Security (TLS) and Secure Sockets Layer (SSL) protocols. It is also a general-purpose cryptography library. In this lab, you will use OpenSSL to encrypt and decrypt text messages.

Note: While OpenSSL is the de facto cryptography library today, the use presented in this lab is NOT recommended for robust protection. Below are two security problems with this lab:

- 1) The method described in this lab uses a weak key derivation function. The ONLY security is introduced by a very strong password.
- 2) The method described in this lab does not guarantee the integrity of the text file.

This lab should be used for instructional purposes only. The methods presented here should NOT be used to secure truly sensitive data.

Required Resources

- Security Workstation virtual machine

Instructions

Part 1: Encrypting Messages with OpenSSL

OpenSSL can be used as a standalone tool for encryption. While many encryption algorithms can be used, this lab focuses on AES. To use AES to encrypt a text file directly from the command line using OpenSSL, follow the steps below:

Step 1: Encrypting a Text File.

- a. Start the Security Workstation VM and log in with username **sec_admin** and password **net_secPW**.
- b. Open a terminal window.
- c. Because the text file to be encrypted is in the **/home/sec_admin/lab.support.files/** directory, change to that directory:

```
[sec_admin@secOps ~]$ cd /home/sec_admin/lab.support.files/  
[sec_admin@secOps lab.support.files]$
```

- d. Type the command below to list the contents of the encrypted **letter_to_grandma.txt** text file on the screen:

```
[sec_admin@secOps lab.support.files]$ cat letter_to_grandma.txt  
Hi Grandma,
```

I am writing this letter to thank you for the chocolate chip cookies you sent me. I got them this morning and I have already eaten half of the box! They are absolutely delicious!

I wish you all the best. Love,
Your cookie-eater grandchild.
[sec_admin@secOps lab.support.files]\$

- e. From the same terminal window, issue the command below to encrypt the text file. The command will use AES-256 to encrypt the text file and save the encrypted version as **message.enc**. OpenSSL will ask for a password and for password confirmation. Provide the password as requested and be sure to remember the password.

```
[sec_admin@secOps lab.support.files]$ openssl aes-256-cbc -in
letter_to_grandma.txt -out message.enc
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
[sec_admin@secOps lab.support.files]$
```

Document the password.

NetSec123!.

- f. When the process is finished, use the **cat** command again to display the contents of the **message.enc** file.

```
[sec_admin@secOps lab.support.files]$ cat message.enc
```

Did the contents of the **message.enc** file display correctly? What does it look like? Explain.

No, it displays unreadable binary characters and symbols. This is encrypted data, not plain text.

- g. To make the file readable, run the OpenSSL command again, but this time add the **-a** option. The **-a** option tells OpenSSL to encode the encrypted message using a different encoding method of Base64 before storing the results in a file.

Note: Base64 is a group of similar binary-to-text encoding schemes used to represent binary data in an ASCII string format.

```
[sec_admin@secOps lab.support.files]$ openssl aes-256-cbc -a -in
letter_to_grandma.txt -out message.enc
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
```

- h. Once again, use the **cat** command to display the contents of the, now re-generated, **message.enc** file:

Note: The contents of **message.enc** will vary.

```
[sec_admin@secOps lab.support.files]$ cat message.enc
U2FsdGVkX19ApWyrn8RD5zNp0RPCuMGZ98wDc26u/vmj1zyDXobGQhm/dDRZasG7
rfnth5Q8NHValEw8vipKGM66dNFyyr9/hJUzCoqhFpRHgNn+Xs5+TOtz/QCPN1bi
08LGTSzOpfkg76XDCK8uPy1hl/+Ng92sM5rgMzLXfEXtaYe5UgwOD42U/U6q73pj
alksQrTWsv5mtN7y6mh02Wobo3A1ooHrM7niOwK1a3YKrSp+ZhYzVTrtksWDl6Ci
XMufkv+FOGn+SoEEuh7l4fk0LIPEfGsExVFB4TGdTizQApRw74rTAZaE/dopaJn0
sJmR3+3C+dmgzZIKEHwsJ2pgLvJ2Sme79J/XxwQVNpw=
[sec_admin@secOps lab.support.files]$
```

Is **message.enc** displayed correctly now? Explain.

Yes, It displays readable ASCII text because it is Base64-encoded, not raw binary.

Can you think of a benefit of having **message.enc** Base64-encoded?

Makes encrypted data safe to transmit via email or text-based systems, Avoids corruption in systems that do not handle binary data properly.

Part 2: Decrypting Messages with OpenSSL

With a similar OpenSSL command, it is possible to decrypt **message.enc**.

- a. Use the command below to decrypt **message.enc**:

```
[sec_admin@secOps lab.support.files]$ openssl aes-256-cbc -a -d -in  
message.enc -out decrypted_letter.txt
```

- b. OpenSSL will ask for the password used to encrypt the file. Enter the same password again.

- c. When OpenSSL finishes decrypting the **message.enc** file, it saves the decrypted message in a text file called **decrypted_letter.txt**. Use the **cat** display the contents of **decrypted_letter.txt**:

```
[sec_admin@secOps lab.support.files]$ cat decrypted_letter.txt
```

Was the letter decrypted correctly?

Yes, The decrypted file matches the original text exactly.

The command used to decrypt also contains **-a** option. Can you explain?

Because the file was Base64-encoded during encryption. The -a option tells OpenSSL to decode Base64 before decrypting.